

Lab0: GitLab

常骏 25300180031

1. 认真阅读文档，学习 Git 的基本用法，并在报告中回答文档中的问题。（15 分）

- 你之前有过多人协同开发的经历吗？如果有，你们是使用什么方式分工协作的？
- 答：有过：1 月的数学建模美赛，使用的是微信文件传输；
- 9 月的数学建模美赛，使用的是 VScode+GitHub，大部分时候使用 vscode 里面的 push 和 pull 就可以完成大部分协作任务，但是仍然需要及时沟通，否则容易出现不同人在本地修改 commit 后，push 时发生冲突的情况。

◦ 思考一下，Git 为什么要设计“暂存-提交”两个步骤？

- 答：我认为 Git 设计“暂存—提交”两步，是因为正式提交前往往需要一个整理和检查修改的中间阶段。

◦ 如果只有“修改—提交”，那么改完文件后，要么一直不提交，要么只能把所有改动一次性记入版本历史。但实际开发中，一次可能改了很多地方，它们未必属于同一个功能。比如一边修 bug，一边顺手改了注释，一边维护了功能 A，一边维护功能 B，可能四个同时进行，然后过了一段时间 A 好了，然后 B 好了。。。。。。这时如果直接提交，相当于四个修改同时提交，但是逻辑上四个修改对项目的贡献内容可能天差地别，把四个完全不同类型的修改打包在一起提交，容易使得提交记录重点不清，阅读者不知道这次修改的重点是什么。

◦ 因此 Git 增加了暂存区。用户可以先挑选已完成、准备提交的修改放入暂存区，不确定或还要继续改的内容留在工作区。这样，暂存区就像提交前的“草稿区”或“候选区”。而且这样可以使逻辑更清晰：即使同时改了 A 和 B，A 先改好，那么就不必把 A 和修改了一部分的 B 同时上传，而是把 A 的改动暂存然后直接上传，这样就能进行一次只修改 A 部分内容的上传，逻辑更清晰，且可以有选择性地只上传已经改动的部分。

◦ 它也提供了检查和撤回空间：某个修改不该进入下次提交，可以从暂存区移除，而不用删除实际修改；提交内容不完整，也可以继续加入其他修改再提交。

◦ 所以，“暂存—提交”把两个问题分开了：哪些修改进入下一次版本？什么时候正式记录为一个版本？

◦ 这种设计让用户可以先组织修改，再生成完整、清晰的提交，而不是把当前所有变化一次性记录下来。

◦ 因此，Git 暂存区的核心意义在于：让用户有选择地组织一次提交，使版本历史更清晰、可控，也让一个提交更准确地对应一个完整的功能、修复或逻辑变化。

◦ git branch 和 git branch -a 的区别是什么？查阅资料并回答。

◦ 答：前者只查看本地分支；后者包括本地分支和远程分支。也就是说，加上 -a（即 --all）选项后，会列出本地分支和远程跟踪分支。

◦ 例如：我自己实验中使用这两个指令：

```
PS C:\Users\crmin\OneDrive\test-git> git branch -a
dev
dev2
main
```

remotes/origin/main

说明带上-a 的结尾可以查到已经 publish 的远程分支

不带上 a:

```
PS C:\Users\crmin\OneDrive\test-git> git branch
```

dev

dev2

- main

(缺少 remotes/origin/main)

- 这说明如果不附带-a, 只能查到本地的分支。

- (另外, 还有一个-r (即--remotes) 远程分支, 只列出远程跟踪分支。)

- **2. 使用此仓库建立个人仓库, 完成 main.c 文件中的 TODO 部分并进行一次 commit。(50 分)**

- 只要填入任意字符串就算完成, 当然你也可以随意发挥(程序的正确性不纳入计分, 有修改即可)。

- 如果你想要编译运行 main.c, 执行

```
make
```

```
./main
```

```
make cl • ean
```

Copy code to clipboard

已完成, 记录存在 main.c 中。

编译 main.c 的 terminal 记录:

```
crmind@crmindnotebook:/ICS-26Fall-FDU.github.io/Gitlab_25300180031_changjun$ make gcc -Wall -O2 -c main.c -o main.o gcc -Wall -O2 -o main main.o
crmind@crmindnotebook:/ICS-26Fall-FDU.github.io/Gitlab_25300180031_changjun$ ./main This is my hw.Cheer up!Hello, world!
crmind@crmindnotebook:~/ICS-26Fall-FDU.github.io/Gitlab_25300180031_changjun$ make clean rm -f main.o main
```

实验过程: 1 git clone 线上仓库

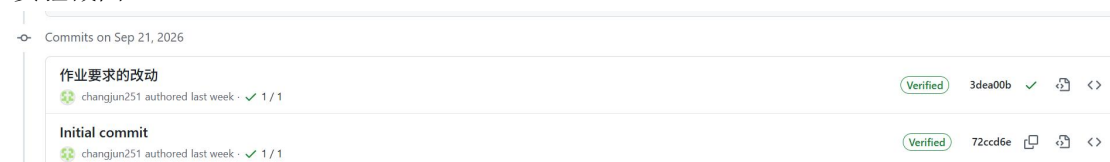
2 cd 切换到线上仓库的本地备份

3 vim main.c 然后点击 i 修改内容, 在 main()函数中增加内容完成后点击:wq,退出了界面.

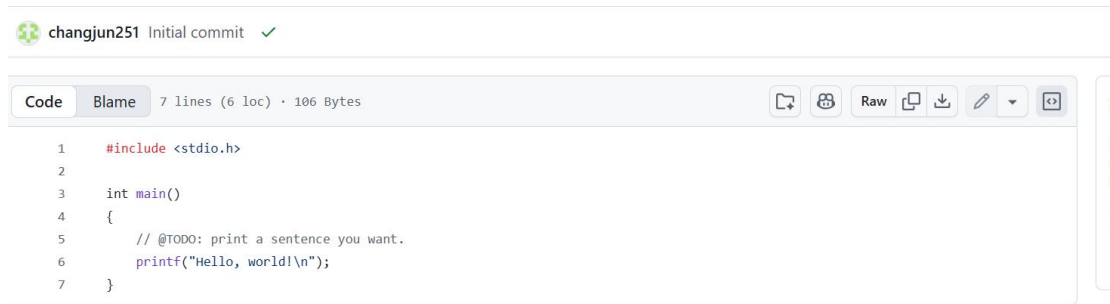
4 git add main.c 暂存修改, 然后 git commit -m "Initial commit".(当时未安装格式检查插件, commit message 未严格遵循规范, 完成任务三后才意识到要符合规范。后续会注意。)

5 git push (由于当时未记录已经没有截图了, 但是肯定成功了, 因为线上的仓库有记录)

实验截图:



Main.c:已经修改完成后的截图:



```
1  #include <stdio.h>
2
3  int main()
4  {
5      // @TODO: print a sentence you want.
6      printf("Hello, world!\n");
7  }
```

3. 在下面的三个网页中任选其二进行阅读，简要概括其内容，并谈谈你对“为什么要学习 Git”这个问题的理解。（15 分）

- Commit Message 规范
- Git Flow 分支控制
- 语义化版本

我选择的是前两个网站。

网站一内容概括：

- 说明了 commit message 的好处：可以列出某次版本以后每次提交的相关信息，方便查看项目历史；也可以用 `grep` 指令查找有某一特征的 commit，快速过滤信息。
- commit 中对 header 的三部分，body 部分，footer 的内容拆解。
- 后续还讲解了撰写合格的 commit 所需的工具：辅助编写工具 `commitizen` 和检查工具 `validate-commit-msg`。4 最后介绍了 changelog，可以生成对不同类型的 commit 的记录和分类文档。

网站 2 内容概括：

- 介绍了主要分支 `master`，`develop` 以及临时分支 `feature`，`release`，`hotfix`，说明了互联网产品运营的整个周期下各个分支是怎么协调合作，上线新功能和修复 bug 的。
- 同时总结了一些合并和产生新分支的规律，例如“从哪里来，回到哪里去”。
- 最后演示了运行一个完整的 `gitflow` 的具体命令集合，介绍了一些 `gitflow` 扩展库，用以标记创建新功能和准备发布新版本（`release`）。

对“为什么要学习 Git”这个问题的理解：

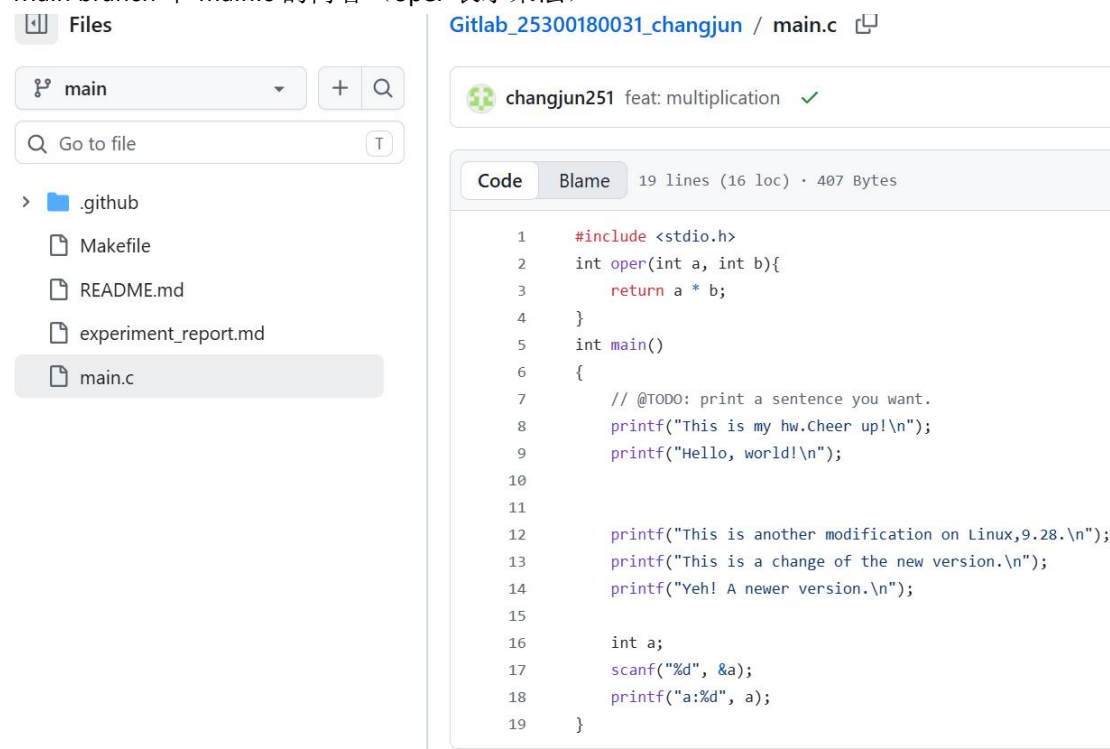
- 为团队协作和中大型项目打好基础：中大型项目往往多人同时开发，如果没有分支管理和版本控制，一个人的改动可能覆盖另一个人的工作，线上出 Bug 也无法快速修补。Git Flow 通过 `feature`、`release`、`hotfix` 等分支划分，让并行开发互不干扰，测试和发布各有独立通道，主分支始终稳定。学 Git 就是学这套协作秩序，知道怎么在团队里安全地加功能、修 bug。
- 提升沟通效率，降低协作成本：格式规范的 message 让任何人通过 `git log` 就能快速了解某次提交的目的，也能按类型过滤提交记录。更重要的是，规范的提交可以直接生成 Change log，发版时不用人工整理各个版本的更新文档，直接 `git` 生成即可，大幅增加效率。
- Git 是开发者回溯之前版本的关键工具：例如开发者发现新版本跑不起来，Git 让他随时回退到可运行的历史版本；想试一个新功能又怕搞坏现有代码，他就可以从 `develop` 切一个 `feature` 分支去尝试，主分支毫发无损。这种安全试错的能力，让开发者敢于重构、敢于探索，而不是因为害怕破坏而束手束脚。

4. 学习 Git 分支管理，新建 feature 分支，在该分支以及 main 分支上对 main.c 分别进行一次修改与提交（10 分）。随后将 feature 分支 merge 到 main 分支（即切换回 main 分支执行 git merge feature`），并处理发生的合并冲突（10 分）。

- 在两个分支上的提交必须要满足：在 main 分支合并时会出现冲突。请你解决这个冲突，并在实验报告里截图表明你遇到并解决了冲突。
- 请阅读 git merge 部分，思考如何修改 main.c 会出现冲突。
- 如果你两次提交之后合并没有出现冲突，不必担心，你可以不用撤回之前的提交，而是继续尝试提交修改并 merge，直到出现冲突并解决。
- 1 对两个分支的分别修改和提交已经完成。
- 2 已经进行了 feature 到 main 的合并，并且发生了冲突，合并了冲突。图片在 experiment_report_picture.pdf 中。
- 具体流程：
- Main branch 中 main.c 的内容（oper 表示乘法）

实验流程记录：

Main branch 中 main.c 的内容（oper 表示乘法）



The screenshot shows a GitLab interface for a repository named "Gitlab_25300180031_changjun". The file "main.c" is selected in the "main" branch. The file content is displayed in a code editor, showing a C program with a multiplication function and a main function. The file is named "main.c" and is located in the "main" branch. The repository is named "Gitlab_25300180031_changjun".

```
1  #include <stdio.h>
2  int oper(int a, int b){
3      return a * b;
4  }
5  int main()
6  {
7      // @TODO: print a sentence you want.
8      printf("This is my hw.Cheer up!\n");
9      printf("Hello, world!\n");
10
11
12      printf("This is another modification on Linux,9.28.\n");
13      printf("This is a change of the new version.\n");
14      printf("Yeh! A newer version.\n");
15
16      int a;
17      scanf("%d", &a);
18      printf("a:%d", a);
19  }
```

产生了新分支 feature

 feature had recent pushes 1 minute ago [Compare & pull request](#)

main 2 Branches 1 Tag

Go to file T Add file <> Code

Switch branches/tags

Find or create a branch...

Branches Tags

main default

feature

View all branches

629cb77 · 9 minutes ago 13 Commits

Initial commit last week

Initial commit last week

local changes last week

Update experiment_report.md 52 minutes ago

feat: multiplication 9 minutes ago

在 feat 分支进行了修改和提交，使得 main 分支里面 main.c 增加了 oper 函数：

changjun251 feat: a new function added ✓

Code Blame

26 lines (20 loc) · 480 Bytes

```
1  #include <stdio.h>
2  int oper(int a, int b)
3  {
4      return a+b;
5  }
```

也在之后对 main 分支进行了修改与提交：

 changjun251 committed yesterday · ✓ 1 / 1

feat: multiplication

 main

1 file changed



Q Search within code



main.c  

... @@ -1,5 +1,7 @@

1 1 #include <stdio.h>

2 -

2 + int oper(int a, int b){

3 + return a * b;

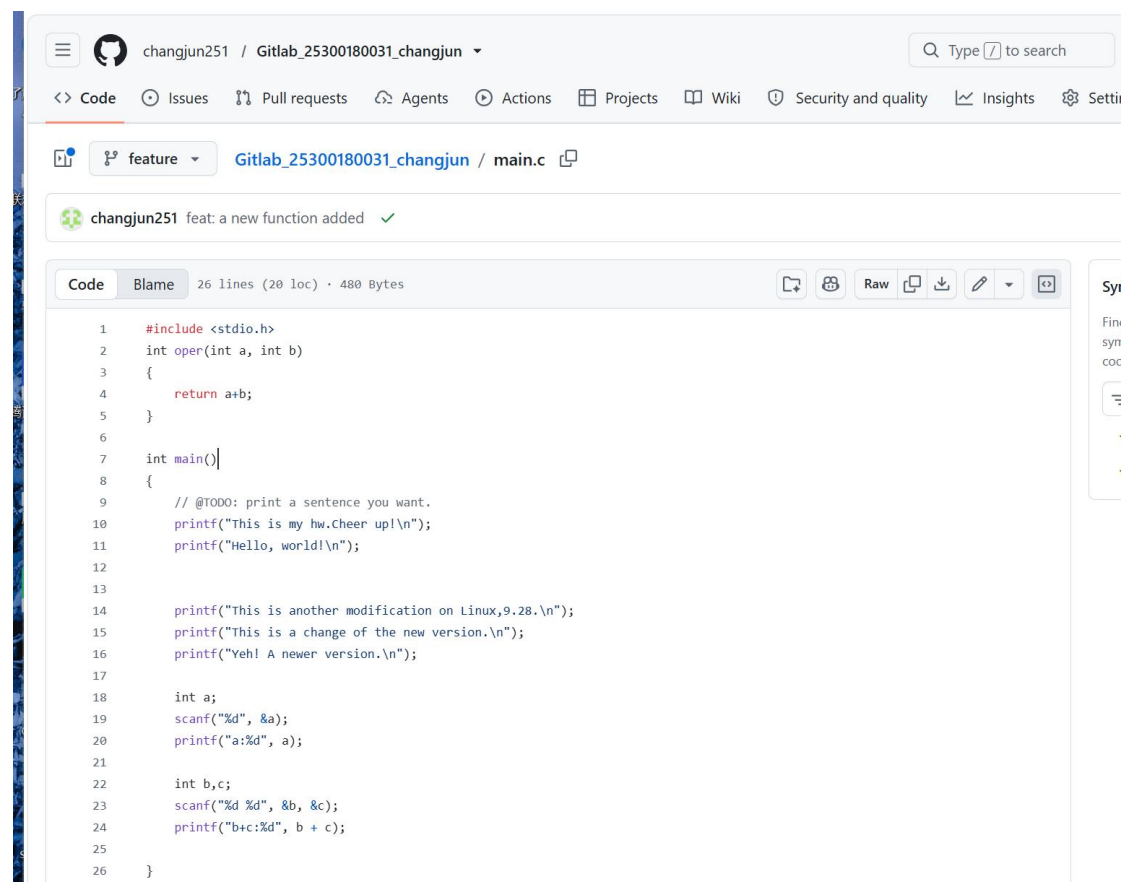
4 + }

3 5 int main()

4 6 {

这之后才进行了合并，具体过程见下面：

Feature 分支里：oper 返回的是加法产生的和，和 main 分支不同。



```
1  #include <stdio.h>
2  int oper(int a, int b)
3  {
4      return a+b;
5  }
6
7  int main()
8  {
9      // @TODO: print a sentence you want.
10     printf("This is my hw.Cheer up!\n");
11     printf("Hello, world!\n");
12
13
14     printf("This is another modification on Linux,9.28.\n");
15     printf("This is a change of the new version.\n");
16     printf("Yeh! A newer version.\n");
17
18     int a;
19     scanf("%d", &a);
20     printf("a:%d", a);
21
22     int b,c;
23     scanf("%d %d", &b, &c);
24     printf("b+c:%d", b + c);
25
26 }
```

遇到冲突的证明：

```
crmind@crmindnotebook:~/ICS-26Fall-FDU.github.io/Gitlab_25300180031_changjun$
git merge feature
Auto-merging main.c
CONFLICT (content): Merge conflict in main.c
Automatic merge failed; fix conflicts and then commit the result.
```

Main.c 中的冲突标记：

```
int main() {
<<<<<<< HEAD
    return a * b;
=====
    return a + b;
>>>>>>> feature
```

后面通过 vim main.c 解决了冲突，把 main 分支里的 oper 从返回 $a*b$ 改成了返回 $(a+b)$ ：

```
#include <stdio.h>
int oper(int a, int b)
{
    return a+b;
}

int main()
{
    // @TODO: print a sentence you want.
    printf("This is my hw.Cheer up!\n");
    printf("Hello, world!\n");

    printf("This is another modification on Linux,9.28.\n");
    printf("This is a change of the new version.\n");
    printf("Yeh! A newer version.\n");

    int a;
    scanf("%d", &a);
    printf("a:%d", a);

    int b,c;
    scanf("%d %d", &b, &c);
    printf("b+c:%d", b + c);
}
```

Git push 之后上传了改动：【这是在 main 分支里的。】

 changjun251 fix: a conflict fixed ✓

Code Blame 27 lines (20 loc) · 481 Bytes

```
1  #include <stdio.h>
2  int oper(int a, int b)
3  {
4      return a+b;
5  }
6
```

- 5. 在 main 分支提交一份实验报告(实验报告单独评分),格式要求为 PDF 或 Markdown。

内容包括：

- 文档中要求回答的问题
- 你的实验步骤
- 必要的截图
- 你的建议（可选）

实验步骤和截图已经在附录和文件中。

建议：问题四的三个网页内容可能过于庞杂，我自己仔细阅读前两个网页然后逐条运行,尝试安装里面涉及的包（例如 `npm`, `validate-commit-msg`, `commitizen`）画了数个小时。建议可以稍微精简一些内容，同时可以对一些难以操作的地方（例如修改 `.json` 相关文件；在 `commitizen/validate-commit-msg` 那一块有很多命令行是直接给出的，可以稍微解释一下它们的意思），或者可以让学生阅读一个教学命令行基本语句的网站。